VISION-LANGUAGE MODELS

Lecture 6

Generative Foundations

Outline

I	**Autoencoders → VAEs**	Make the latent space sampleable so we can generate.
II	**GANs**	Generator vs Discriminator : learn to generate by competition.
III	**Diffusion**	Add noise, then learn to remove it, used in modern image generators.

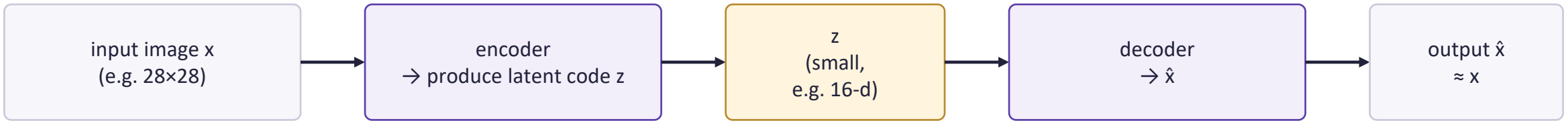
From “understanding” to “generating”

- Everything thus far was understanding:
  - image/video in → text out (describe, answer, ground, locate)
- Now we move in the opposite direction:
  - something in → a new image out aka generation
- What “generate” asks of a model:
  - Not “copy a training image” — that's memorization
  - We want new images that look like they could have been in the training set, “similar distribution”
  - Example: train on 10,000 photos of faces → produce a face of a person who has never existed, that still looks real/plausible
- The core question we consider here:
  - How do you learn the “look” (distribution) of a whole category of images well enough to synthesize convincing new members of it?

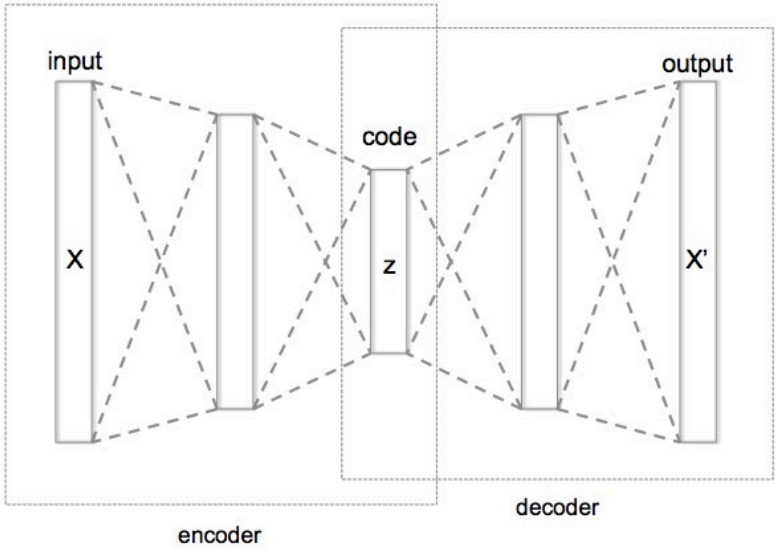
PART I

From autoencoders to VAEs

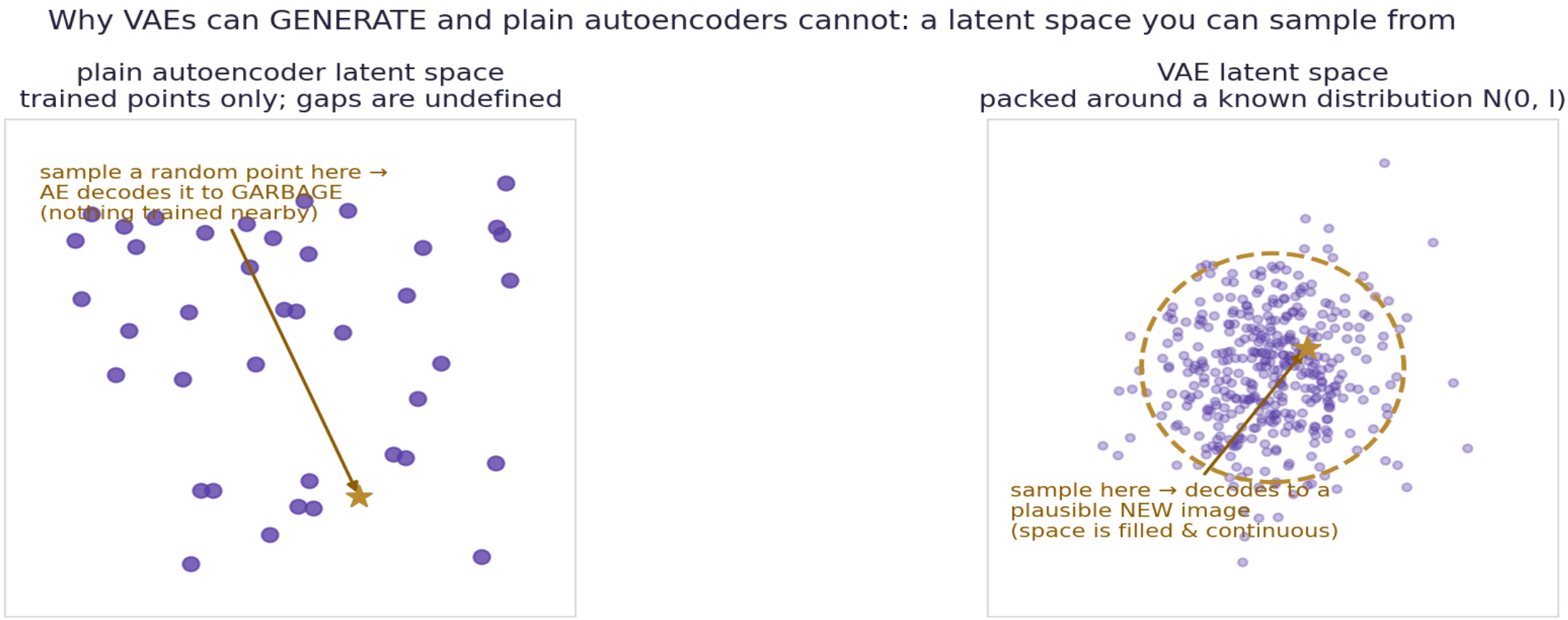
Recap: the autoencoder



- What we want: a generator – something that can produce new outputs that are similar to input (for image generation later)
- AE trained to reconstruct input: make x̂ as close to x as possible.
  - Self-supervised: Loss = reconstruction error, e.g. mean-squared error $\|x - \hat{x}\|^2$
- The code z is a compressed summary — the "bottleneck" forces the network to keep only what matters
- In theory, one can generate new outputs by picking a random z and decoding…. Fails! Why?
  - an autoencoder only ever learned to decode the specific z-codes its training images mapped to
  - The space between/around those codes was never trained/defined –
    - If e.g. (1,1) and (10,10) exist as encoded training examples…what does (5,5) reconstructed mean?
  - Thus, decode a random z and you usually get noise or garbage.



Why a plain autoencoder can’t (usefully!) generate



- Left: the training codes sit at scattered spots with big empty gaps. Right: a VAE forces the codes into a distribution you can safely sample from.

Figure: Claude.

Variational Autoencoders

- Main insight: the encoder outputs a probability distribution over codes, not a single code
  - We can now talk about a continuous latent space
  - For each image **x** it predicts a mean $\boldsymbol{\mu}(\mathbf{x})$ and a standard deviation $\sigma(\mathbf{x})$, which together define a Gaussian $q(z|x) = N(\mu, \sigma^2\mathbf{I})$ — the approximate posterior: given this image, where could its code plausibly lie in our latent space?
  - During training we draw a sample $z \sim q(z|x)$ and decode that, so the decoder has to handle a whole region of latent space rather than one isolated point
- Plus a constraint: every posterior is pulled toward the same prior $p(z) = N(0, I)$
  - The same target for every image, so the posteriors overlap and together cover the region we will later sample from
  - Without that pull the encoder could place posteriors wherever it liked — and the gaps from the previous slide come back
- Generation then costs very little:
  - Only use decoder part of network
  - draw $z \sim N(0, I)$, decode it, and you have a brand-new image

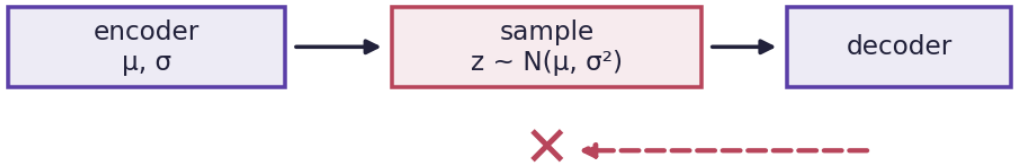
encoder output:
$\mu(x)$ the mean
$\sigma(x)$ the std. deviation

$\mathbf{q(z|x) = N(\mu, \sigma^2 I)}$

Reparameterization trick: fix backpropagation in VAE

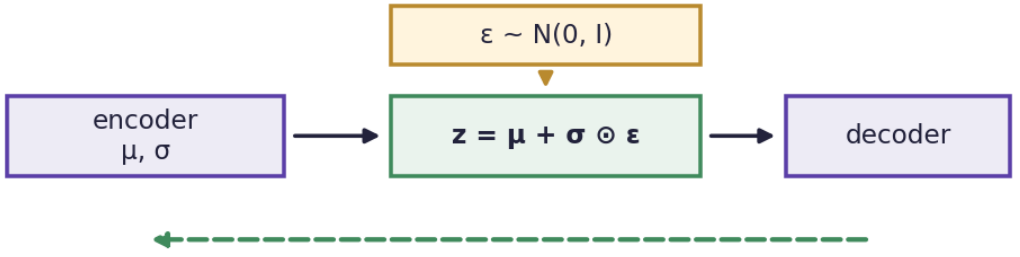
Same distribution, different route: if $\varepsilon \sim N(0, I)$ then $\mu + \sigma \odot \varepsilon \sim N(\mu, \sigma^2 I)$

Naive version: sampling blocks backpropagation



gradient cannot flow back through a random draw — the encoder receives no learning signal

Reparameterised: the randomness moves off the path



the randomness is now an INPUT, so z is a differentiable function of μ and σ — gradients reach the encoder normally

The problem: training needs ∂loss/∂μ and ∂loss/∂σ, but "draw a random sample" is not a differentiable operation. With z drawn inside the network, no gradient ever reaches the encoder.

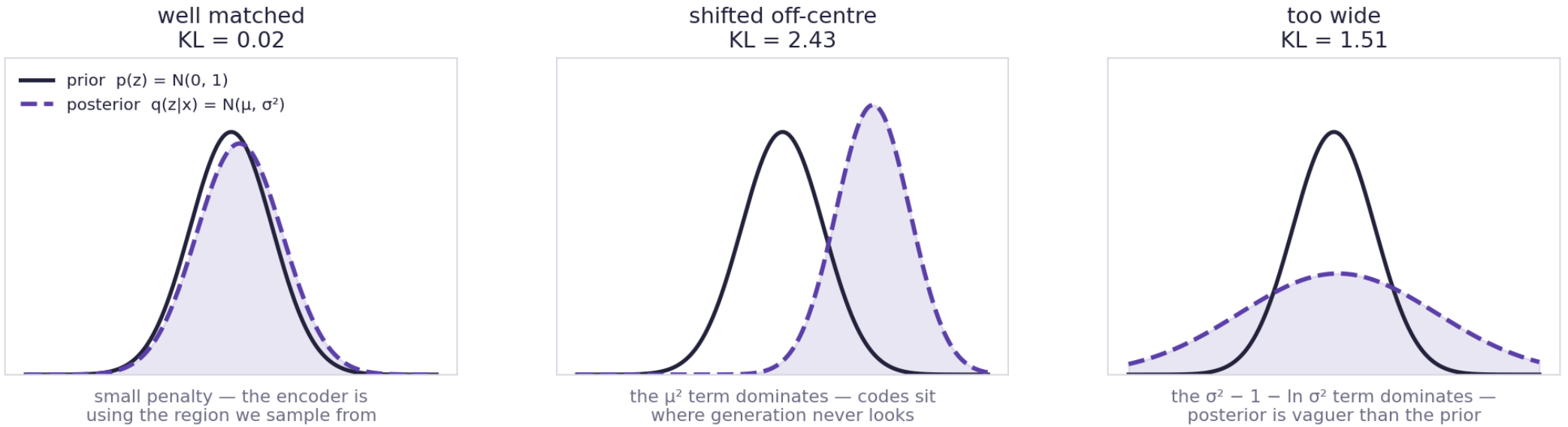
The fix: draw $\varepsilon \sim N(0, I)$ — noise that depends on no parameter — and set $z = \mu + \sigma \odot \varepsilon$. The distribution of z is unchanged, but z is now a differentiable function of μ and σ, so gradients flow to the encoder as usual.

This is what makes a VAE trainable end-to-end.

Kingma & Welling 2014, §2.4 (the SGVB estimator / reparameterisation trick). Figure: Claude

The KL divergence: comparing two distributions

KL(q ‖ p): how far the encoder's posterior sits from the prior — zero only when they are identical, and never negative



- **What it measures:** KL(q ‖ p) is the gap between a distribution q and a reference p. It is never negative, and is exactly zero only when q = p. It is not symmetric — here we always measure the encoder's posterior against the prior.
- **KL formula, for the Gaussian case we use — per latent dimension, q = N(μ, σ²) against p = N(0, 1):** $KL = \frac{1}{2}(\mu^2 + \sigma^2 - 1 - \ln \sigma^2)$. The μ^2 term penalises being off-centre; $\sigma^2 - 1 - \ln \sigma^2$ penalises being too wide or too narrow, and is zero exactly at $\sigma = 1$.
- **Why KL term is needed:** it is the only thing tying each image's posterior to the distribution we actually sample from at generation time.

Kingma & Welling 2014, App. B (closed-form Gaussian KL). Figure: Claude.

The VAE objective: reconstruction + KL

$$\text{loss} = E[\text{ reconstruction error }] - KL(\, q(z|x) \,\|\, p(z)\,)$$

"rebuild x from a sampled code" "and keep the posterior near the prior"

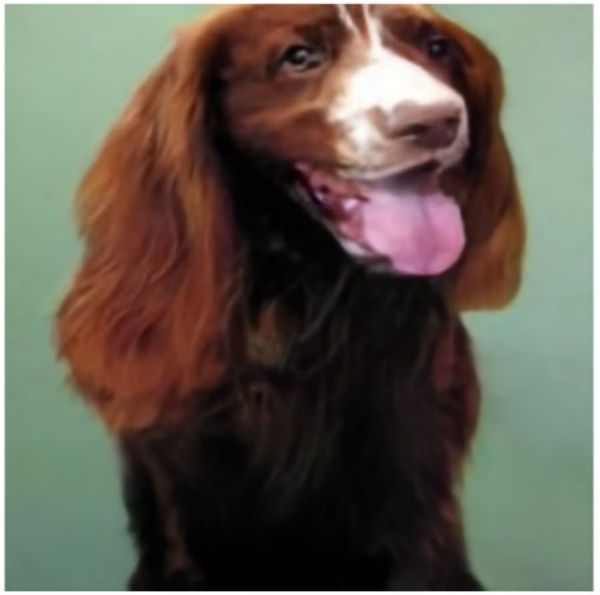
- A common variant is the β-VAE – this allows more control via a parameter β
- Note this part is a somewhat more hand-waving explanation, else we need a digression into stats….
- Loss is the negative ELBO — the **E**vidence **L**ower **BO**und
- The first term usually collapses to a simple MSE loss (generated vs input) for images - yay!
- Maximising the ELBO and minimising the negative ELBO are the same thing (both used).

  The two loss terms pull in opposite directions:

  – Reconstruction alone → σ shrinks toward 0 and the posteriors drift apart: you are back to a plain autoencoder, gaps and all

  – KL alone → $q(z|x) = N(0, I)$ for every image, so the code carries no information about x. This is posterior collapse: a perfect match to the prior, and useless reconstruction
- Balanced → posteriors close enough to the prior to sample from, distinct enough to rebuild each image faithfully

VAEs: what to remember



- A VAE is an autoencoder whose encoder outputs a distribution rather than a point, regularised so that those posteriors together match a prior you can sample from
- Strengths:
  - Stable, simple training (one network pair, one clear objective)
  - A smooth, well-structured latent space — nearby codes decode to similar images, which is what makes interpolation and editing work
- Weakness that matters for images:
  - VAE samples tend to be blurry — the reconstruction term averages over everything the posterior could have encoded, which smooths away fine detail



- Why VAEs are important:
  - They rarely generate final images today but the VAE's **encoder** is core part of Stable Diffusion
    - it compresses images so diffusion runs cheaply…

Kingma & Welling 2014.

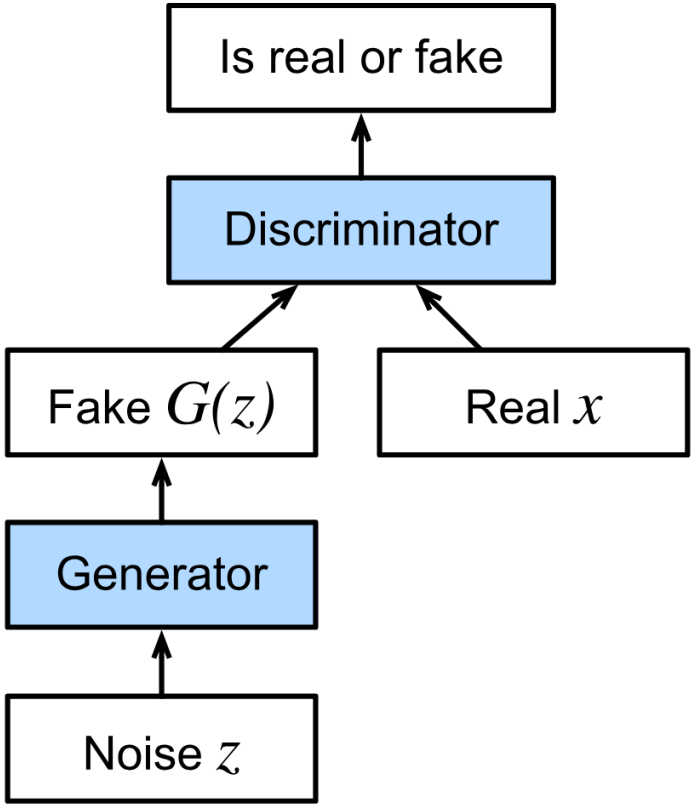
PART II

GANs – generative adversarial networks

GANs: a forger vs a detective

- **Two networks with opposing objectives, trained together:**
  - Generator (the forger): takes random noise → tries to produce a realistic fake image
  - Discriminator (the detective): takes one image → guesses "real" (from the dataset) or "fake" (from the generator)
- We use noise as a seed → image diversity (vs a single value that maps to same output always)
- **The networks improve by competing:**
  - The discriminator is trained to catch fakes; the generator is trained to deceive the discriminator
  - As the discriminator improves, the generator is forced to make better fakes — and vice versa: an "arms race"
- **At the end you keep only the generator — now good enough that its fakes pass as real**





Images: Largescale GAN…Brock etal, .arXiv:1809.11096; wikipedia

GAN objective

discriminator: network D(x) outputs probability image x came from real data (not generator)
train D to maximize output value
and simultaneously
Train Generator network G(z) to minimize log(1 – D(G(z)))

- This is a 2-player “minimax” game (one side maximizes what the other minimizes) - well studied optimization problem
- These two neural networks, D and G, are trained in tandem through a minimax objective that combines them
- **Intuition:**
  - Discriminator sees a real photo, says 0.9 “real” ✓ but then sees a fake, says 0.8 “real” ✗ (fooled)

    → discriminator adjusts weights to score that fake lower next time
  - Generator is told “the discriminator still flagged 30% of your fakes” → adjusts weights to push those cases toward “real” category
  - Repeat millions of times → fakes become indistinguishable from real
- **Why training is notoriously hard:**
  - It's a moving target: both generator and discriminator networks change at the same time (in practice they may alternate training)
  - If the discriminator gets too good too quickly, the generator gets no useful training signal (“you're always wrong” teaches nothing).
  - Balancing the two is the whole art (mode collapse, instability).

Goodfellow et al. 2014, the minimax objective.

GANs: what to remember

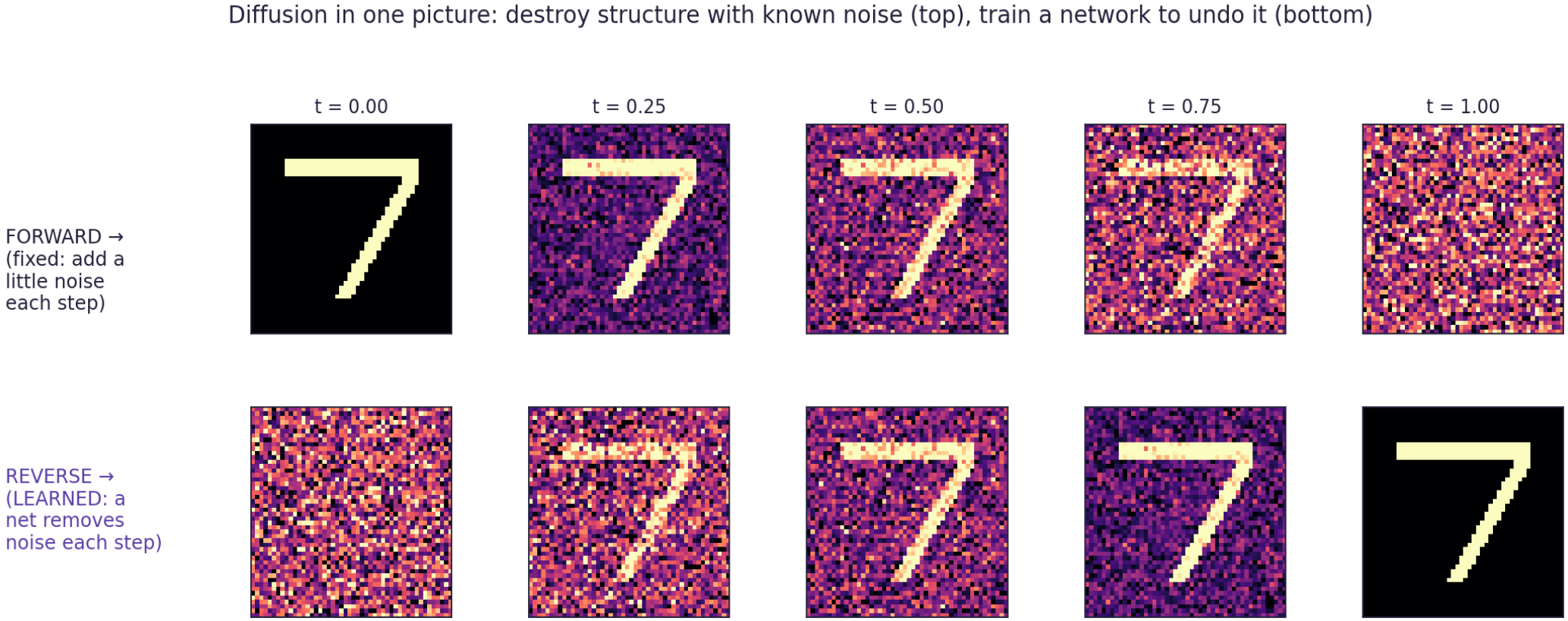
- **Core idea:** train a generator by having it try to fool a detector that's simultaneously learning to catch it out
- **Strengths:**
  - SHARP, detailed images — no blurry averaging like VAEs (the competition rewards *crisp, convincing detail*)
  - Fast to sample: one forward pass through the generator = one image
- **Weaknesses:**
  - Unstable, finicky training; can enter "mode-collapse" (produce only a few kinds of image)
  - No well-structured latent space or likelihood — harder to control and evaluate
- Historically: GANs ruled image generation ~2016–2021.
- Many complex variants to inject generation diversity, condition generation on an image, produce images in specific style etc
- Then diffusion matched their sharpness with far more stable training and greater diversity — and took over.
- GANs are still used, but not very common – they can generate quickly and the discriminator idea is used in some other networks



PART III

Diffusion

Diffusion – core idea



- Top (forward): start from a real image, add a little random noise repeatedly, until image is noise only. This part is fixed — no learning, just a simple process for adding noise.
- Bottom (reverse): train **one** network to undo **a single step** of that noise. Run it many times from pure noise input

  → a new de-noised image appears.

Ho et al. 2020 (DDPM), arXiv:2006.11239.

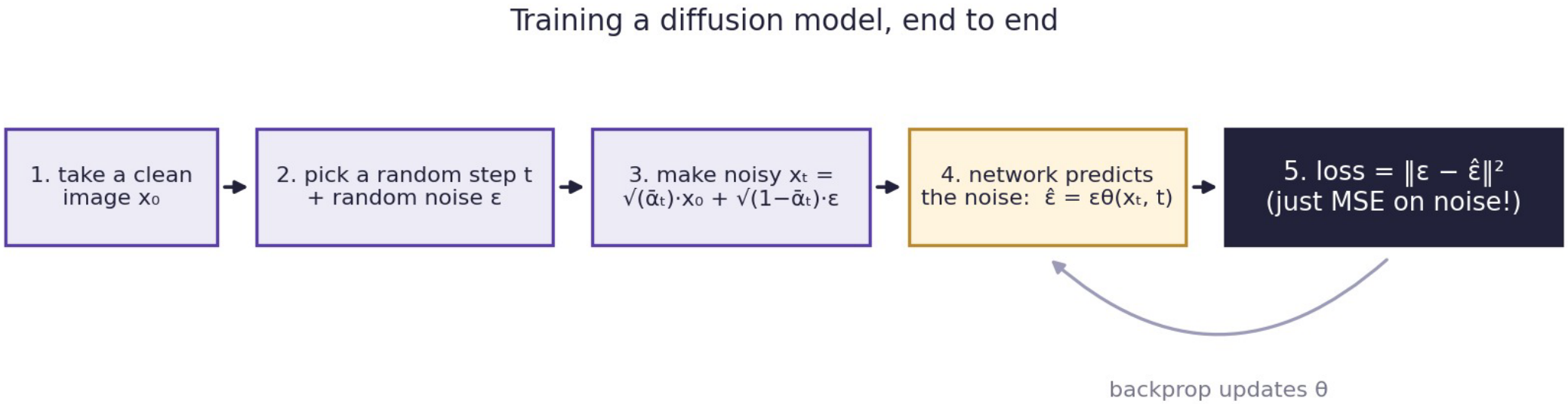
Why break it into incremental steps?

- Generating a whole image from pure noise in one shot is extremely hard — that's essentially the GAN's difficulty
- Diffusion's trick: Rather solve a very small, “easy” problem — “this image is slightly noisy; make it slightly less noisy”
  - Each denoising step is a small, well-behaved regression the network can learn reliably
  - Chaining hundreds of easy steps accomplishes what one giant leap could not
- An analogy: you can't sculpt a statue in one strike, but you can remove one chip at a time. Diffusion removes one “chip” of noise per step
- This “many easy steps beat one hard step” is exactly why diffusion trains stably where GANs struggle — and why it produces both sharp and diverse images



Ho et al. 2020; Sohl-Dickstein et al. 2015 (the original diffusion idea); Image: generated samples from Ho et al 2020.

How it's actually trained: predict the noise



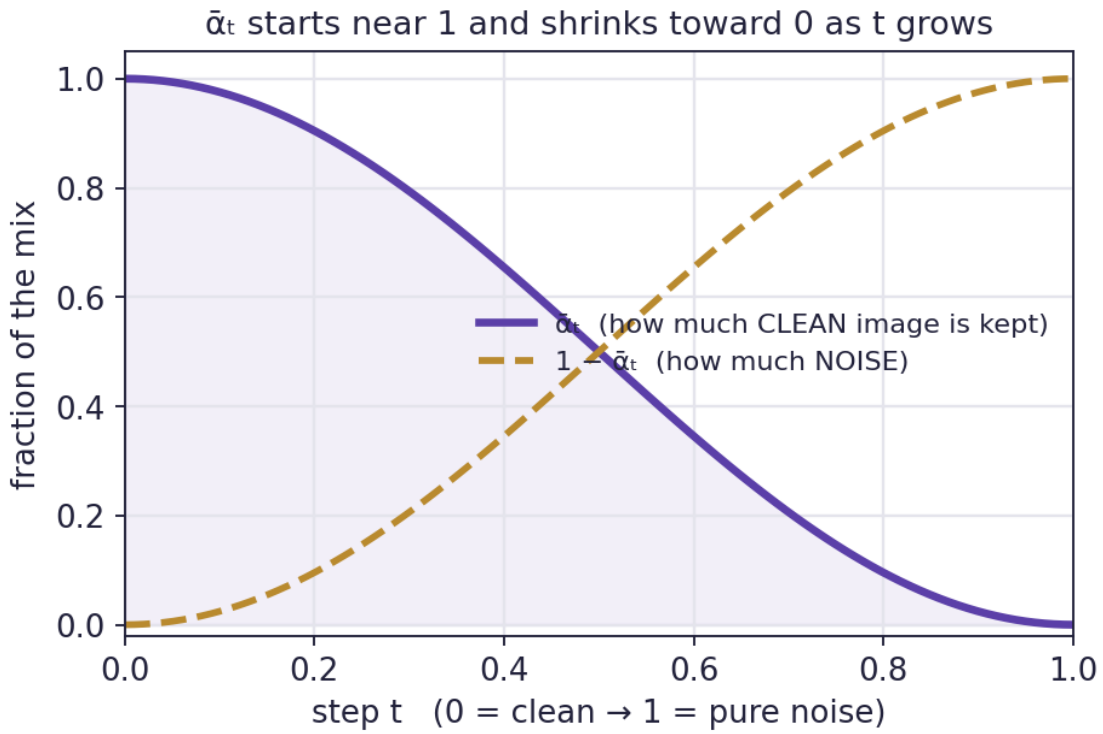
The whole training objective: guess the noise that was added. No likelihood, no adversary — one clean regression.

- NOTE: the network is **never** asked to make an image. It's asked to guess the noise that was added to a slightly-noised image — a plain regression.
- Loss = mean-squared error between the real noise ε and the network's guess ε̂. No adversary (GAN), no distributions to match by hand (VAE) — one simple, stable objective.

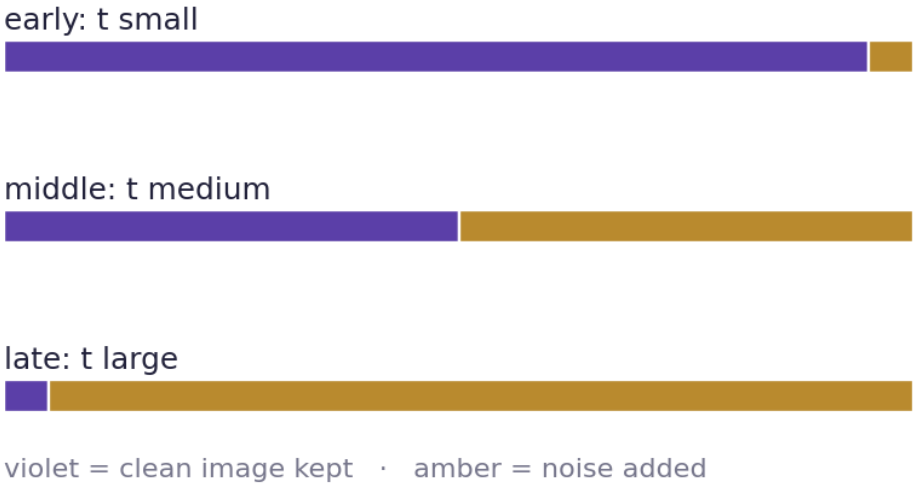
Ho et al. 2020, the simplified noise-prediction (ε-prediction) training objective.

A closer look: what is $\bar{\alpha}_t$ in that formula?

What $\bar{\alpha}_t$ is: the dial that sets how much clean image vs noise is in x_t



$$x_t = \sqrt{\bar{\alpha}_t} \cdot (\text{clean } x_0) + \sqrt{(1-\bar{\alpha}_t)} \cdot (\text{noise } \varepsilon)$$



- $\bar{\alpha}_t$ ("alpha-bar at step t") is just a number between 0 and 1 that says how much of the clean image survives at step t. It comes from the fixed noise schedule — we choose it in advance, the network never learns it.
- It shrinks as t grows: $\bar{\alpha}_t \approx 1$ early (mostly clean image) → $\bar{\alpha}_t \approx 0$ late (almost pure noise). So $\sqrt{\bar{\alpha}_t} \cdot x_0$ fades out while $\sqrt{(1-\bar{\alpha}_t)} \cdot \varepsilon$ fades in — that's how x_t smoothly turns an image into pure noise.

Ho et al. 2020 ($\bar{\alpha}_t$ = cumulative product of the noise schedule).

The diffusion loss

$$\text{loss} = \| \varepsilon - \varepsilon_\theta(x_t, t) \|^2 \quad \text{(mean-squared error)}$$

the actual noise we added — *the network's prediction of it*

- **The loss components:**
  - ε (epsilon): the random noise we added to a clean image during the forward process. We know it exactly since we chose it
  - x_t: the resulting noisy image at step t (built with $\bar{\alpha}_t$ from the previous slide). This is what the network sees as input
  - $\varepsilon_\theta(x_t, t)$: the network's prediction of that noise, given the noisy image and which step t it is (θ = the network's weights)
  - $\| \cdot \|^2$: squared difference — standard regression error.

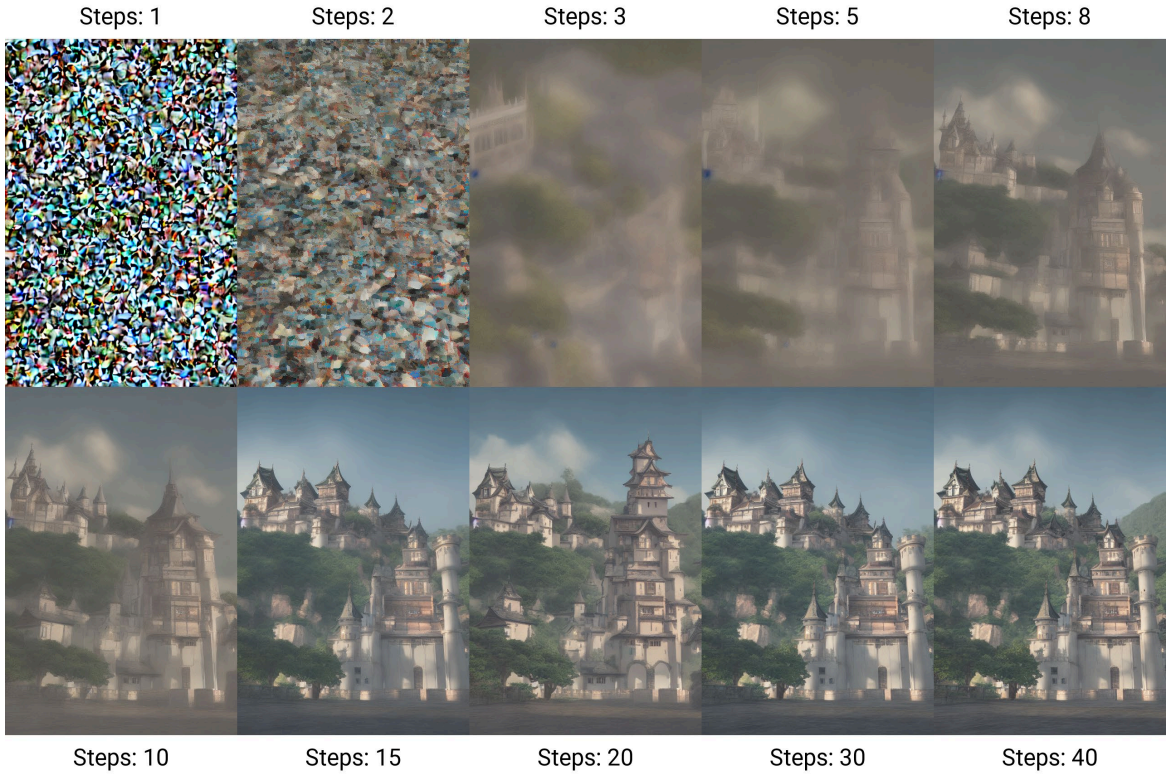
- Compare the complexity of three losses you've seen:
  - VAE: reconstruct accurately + sample latent space in a useful way (to generate diverse images)
  - GAN: fool an adversary (a complicated moving target, often unstable)
  - Diffusion: predict the noise (a simple regression)

Ho et al. 2020 (the L_simple objective).

Generating an image, step by step

- **Once the noise-predictor is trained, sampling is a loop:**
  - 1. Start from pure random noise (this is “z” — like the VAE, generation begins with noise)
  - 2. Ask the network: “what noise is in this?” → subtract a bit of it → a slightly cleaner image
  - 3. Repeat, say, 50–1000 times → a fully formed, novel image emerges from the initial random noise

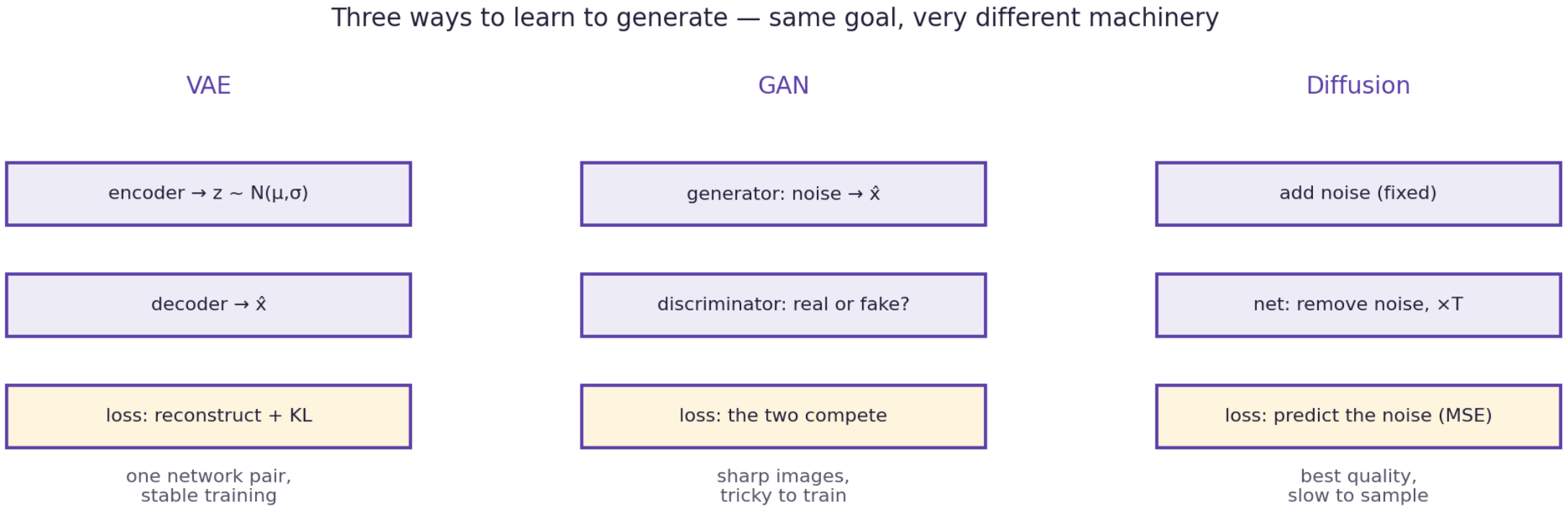
- **Pros and cons:**
  - Quality and diversity are excellent and why diffusion won
  - Speed: slow — a VAE or GAN makes an image in ONE pass; diffusion needs many. This “many steps” cost is the main practical drawback (and a big research topic: fast samplers cut 1000 steps to ~4)



- For high-res image generation, diffusion is too expensive.
- The fix (run diffusion in a VAE's compressed space) yields Stable Diffusion

Ho et al. 2020 (sampling loop); fast samplers e.g. Song et al. 2021 (DDIM); image: wikipedia

Comparison of image generators



- Many GAN variants exist – e.g. conditioned GANs, cycleGANs etc, but all suffer from output diversity or training stability issues
- The dominant image generation approach is now diffusion — we will look at how to make it faster and condition it on text later.

Summary

VAE

An autoencoder whose encoder outputs a distribution q(z|x), not a point. Loss = reconstruction error + KL(q ‖ prior); the reparameterisation trick makes it trainable. Smooth latent, blurry samples.

GAN

A forger (generator) vs a detective (discriminator), improving by competition. Loss= competition between the networks. Sharp images, one-pass sampling, but famously unstable training.

Diffusion

Add noise by a fixed recipe; train one network to predict-and-remove it, then repeat from pure noise initialisation. Loss = mean-squared error on the noise — simple regression. Best quality and diversity; slow to sample. The basis of every modern generator.

In closing…

Diffusion won by turning very complex problem into many easy steps — stable to train, excellent to look at. Next - how to make it efficient and drive it with text.

Resources

- **Key papers**
  - Kingma & Welling (2014). Auto-Encoding Variational Bayes (the VAE). arXiv:1312.6114
  - Goodfellow et al. (2014). Generative Adversarial Networks. arXiv:1406.2661
  - Ho, Jain & Abbeel (2020). Denoising Diffusion Probabilistic Models (DDPM). arXiv:2006.11239
  - Higgins et al. (2017). β-VAE (the KL-weight knob) · Dhariwal & Nichol (2021). Diffusion beats GANs · Song et al. (2021). DDIM (fast sampling)
- **Best way to build intuition (recommended over more papers)**
  - Hugging Face Diffusion Course, unit 1 — huggingface.co/learn/diffusion-course
- **To try out:**
  - Load a small pretrained diffusion model with HuggingFace diffusers and generate one image; then observe the intermediate denoising steps